Program: 1

Name: Pushkar Poman

Div: B

Roll no: B22069

Problem Statement: Design and implement Parallel Breadth First Search and Depth First Search based on existing algorithms using OpenMP. Use a Tree or an undirected graph for BFS and DFS.

Code:

```
#include <iostream>

#include <vector>

#include <queue>

#include  <stack>  #include

<omp.h> using namespace

std;  class  Graph  {  int  V;

vector<vector<int>>    adj;

public:

   Graph(int V) { this->V

      = V; adj.resize(V);

   }

   void      addEdge(int      u,      int      v)      {

      adj[u].push_back(v);

      adj[v].push_back(u); // undirected graph

   }

   // ◆ Parallel BFS
```

```
void    parallelBFS(int    start)    {
   vector<bool> visited(V, false);
   queue<int> q; visited[start] =
   true;

   q.push(start);    cout    <<
   "\nParallel  BFS:  ";  while
   (!q.empty())  {  int  size  =
   q.size();

      #pragma omp parallel for
      for (int i = 0; i < size; i++) { int
         node;

         #pragma omp critical
         {
            if (!q.empty()) { node
               = q.front();

               q.pop();    cout    <<
               node << " ";

            }
         }
         for (int neighbor : adj[node]) { if
            (!visited[neighbor]) {

               #pragma omp critical
```

```
            {
                if (!visited[neighbor]) { visited[neighbor]
                    = true;

                    q.push(neighbor);
                }
            }
        }
    }
  }
}
// ◆ Parallel DFS (using stack)
void parallelDFS(int start) {
vector<bool> visited(V, false);
stack<int> st; st.push(start);
cout << "\nParallel DFS: "; while
(!st.empty()) { int node;

    #pragma omp critical
    {
        if (!st.empty()) {
            node = st.top();
            st.pop();

        }
```

```
        }
        if    (!visited[node])    {
          visited[node] = true;
          cout << node << " ";

          #pragma omp parallel for
          for (int i = 0; i < adj[node].size(); i++) {
            int    neighbor    =    adj[node][i];    if
            (!visited[neighbor]) {

                                        #pragma omp critical
              {
                                          st.push(neighbor);
              }
            }
          }
        }
      }
  }
};
int main() { int
  V = 6;

  Graph    g(V);    //    Sample
  undirected graph

  g.addEdge(0, 1);
```

```
    g.addEdge(0, 2);

    g.addEdge(1, 3);

    g.addEdge(1, 4);

    g.addEdge(2, 5);

    g.parallelBFS(0);

    g.parallelDFS(0);

    return 0;

}
```

Output:

```
$

...

dell@Pushkar MINGW64 ~/Videos/BE Practicals
$ cd 'c:/Users/dell/Videos/BE Practicals/HPC'

dell@Pushkar MINGW64 ~/Vides/BE Practicals/HPC
$ cd "c:\Users\dell\Videos\BE Practicals\HPC\" && g++ Program1.cpp -o Program1 && "c:\Usex\dell\Videos\BE Practicals\HPC\
"Program1
bash: cd: too many arguments

dell@Pushkar MINGW64 ~/Vides/BE Practicals/HPC
$ g++ Program1.cpp -o Program1
./Program1

Parallel BFS: 0 1 2 3 4 5
Parallel DFS: 0 2 5 1 4 3
dell@Pushkar MINGW64 ~/Videos/BEPracticals/HPC
$
```